

Assignment 4: Build, Measure, and Explain CPU Sharing and Parallel Data Processing

Linux Systems and Performance - Week 4

Parallel Data Engine -> cgroup Resource Manager -> User-Space Scheduler

1. Assignment Philosophy

This assignment is not evaluated only by whether the code runs. You will build a parallel data engine, control its CPU resources, reuse its block-processing function in a user-space scheduler, and explain the measured behavior using the Week 4 execution model.

PREDICT -> BUILD -> MEASURE -> ANALYZE -> EXPLAIN

Important: All coding tools may be used. However, you are responsible for understanding, debugging, modifying, and explaining all submitted code and experimental results. Implementation alone is not sufficient for full credit.

2. Challenge Levels

Level	Build	Load / Trigger	Measure / Observe	Main Question
Level 1	parallel data engine (dengine)	threads, block size, counter layout	elapsed time, GB/s, per-thread CPU time	How do locality and false sharing affect parallel work?
Level 2	cgroup CPU manager (cpumgr)	live weight/quota changes; cpuset	cpu.stat, schedstat, blocks/s	How does Linux share and limit CPU time?
Level 3	FIFO / RR / weighted RR scheduler (usched)	quantum, block size, job mix	response time, CPU share, overhead	How do fairness and preemption affect short jobs?

Levels are cumulative. Reuse the same data and work_block() implementation so that comparisons are meaningful. Absolute performance across different machines is not graded. Kernel modification is not required.

3. Common Interface and Rules

- Use C for the required implementation unless otherwise approved. Run on your laptop with WSL2 kernel 6.6, systemd=true, and cgroup v2 cpu/cpuset controllers enabled. Use two vCPUs (processors=2) unless otherwise specified.
- Build all programs with -g -fno-omit-frame-pointer. Use elapsed time to compare cache effects; hardware counters are unavailable in the required WSL2 setup. Keep machine, build options, input, and competing load fixed.
- Fixed parameters: N = 64 MB of 32-bit records; a transformed record passes when its value modulo 7 equals 3. Short jobs in Level 3 use 4 base blocks of 256KB (1MB). Everyone uses the same values, so results are comparable across reports.
- You may modify the skeleton code as needed to complete the assignment.
- Predict before running each experiment. Repeat measurements. Record exact commands, PID, uname -r, and execution time in raw logs saved with script -c or tee. Every measurement row must identify its source log.
- Record the sum and count of passing records on every run (in the CSV for usched). Results must match across configurations for the same job lengths. Remove experiment cgroups after normal exit and Ctrl-C.
- Document kernel-version-dependent details when they affect the implementation or observed behavior. Screenshots alone are not sufficient; include short interpretations and cite your own measurements.

4. Required Interface and Test Tools

```
./dengine -n <MB> -t <threads> -m <stage|block> -b <block_KB>
          [-s packed|padded] [-u record|block] [-r repeats] [-P]
sudo ./cpumgr [--weight W] [--max QUOTA PERIOD] [--cpuset CPUS]
          [--at SEC:weight=W | SEC:max=Q,P] [--bg WEIGHT] -- ./dengine ...
./usched -p <fifo|rr|wrr> -q <quantum_ms> -b <block_KB> -w <workers> -j <jobfile> [-o <path>]
```

In the commands below, replace N with 64. With --bg, cpumgr provides an internal two-thread CPU burner. Use ps -L, cgroup statistics, and per-thread schedstat to validate runtime behavior.

5. Level 1 - Parallel Data Engine

5.1 Required Implementation

- Fill the input with a fixed pseudo-random pattern. Apply the same arithmetic transform to every record, filter the transformed values, and compute the sum and count of passing records.
- In stage mode, threads divide the full array, finish the transform, and synchronize at a barrier. They then filter the array, synchronize again, and aggregate the results.
- In block mode, divide the array into blocks of -b KB. Each worker obtains a block index through a shared atomic fetch-add counter and performs all three stages on that block before taking the next block. Reuse this work_block() function in Level 3.
- Maintain per-thread counters for processed records, passing records, and sum. In packed layout, place all counters next to one another in one structure. In padded layout, give each thread a separate slot aligned to 64 bytes.
- In record update mode, update counters for every record. In block update mode, accumulate locally and update counters once per block. Sum all thread counters to obtain the final result.
- Before measurement, print PID and all TIDs and name threads worker-N. At exit, print elapsed time in ms (best of repeats), input throughput in GB/s, per-thread CPU time from RUSAGE_THREAD, voluntary/involuntary context switches, and aggregate results.

5.2 Required Experiments

```
for t in 1 2 4 8; do
  ./dengine -n N -t $t -m stage -r 3
  ./dengine -n N -t $t -m block -b 256 -r 3
done

for b in 4 16 64 256 1024 4096; do
  ./dengine -n N -t 2 -m block -b $b -r 3
done

for s in packed padded; do
  for u in record block; do
    ./dengine -n N -t 2 -m block -b 256 -s $s -u $u -r 3
  done
done
```

Report a compact comparison table. Add the exact configuration and source-log path to each row.

Comparison	Prediction	Elapsed / GB/s	CPU Time / Switches	Interpretation
stage vs. block; 1/2/4/8 threads				
4KB-4MB blocks; 2 threads				
packed/record vs. padded/record				
padded/record vs. padded/block				

5.3 Required Explanation

1. With two threads, which mode was faster, and by how much? Explain using working-set size and data movement through the L2/L3 cache hierarchy.
2. At what block size did increasing the size stop reducing elapsed time? Compare it with the L2 cache size observed in Lab 4. Which cost dominates with 4KB blocks?
3. Quantify packed/record versus padded/record and explain cache-line ownership transfers between CPUs. Then compare padded/record with padded/block. Which cost remains after false sharing is removed, and why does local accumulation reduce it?
4. Did any eight-thread configuration outperform two threads on two vCPUs? Use per-thread CPU time and involuntary context switches to explain the result.

6. Level 2 - CPU Resource Manager

6.1 Required Implementation

- Create `/sys/fs/cgroup/cpumgr/<manager-pid>` and enable `cpu/cpuset` in the parent subtree. Apply `--weight` to `cpu.weight`, `--max` to `cpu.max`, and `--cpuset` to the allowed CPU list.
- Fork a child that writes its own PID to `cgroup.procs` before executing `engine`. With `--at SEC:...`, rewrite the selected setting SEC seconds after execution starts. The same run must show behavior before and after each change.
- Every second, record deltas of `cpu.stat` `usage_usec`, `nr_throttled`, and `throttled_usec`, plus summed per-thread `schedstat` `execution/run_delay` deltas. Convert times to ms; record `interval_ms`, `cpu.weight`, `cpu.max`, and `cpuset.cpus.effective`.
- Use `engine -P` to print completed blocks, blocks/s, and timestamps for each one-second interval. Align these samples with `cpumgr` logs to compare throughput with CPU policy changes.
- With `--bg WEIGHT`, run the internal two-thread burner in `/sys/fs/cgroup/cpumgr/bg-<manager-pid>` with that weight. Weight is relative to competing sibling groups. Clean up both groups on exit, including `Ctrl-C`.
- At startup, `engine` must print online CPU count from `sysconf(_SC_NPROCESSORS_ONLN)` and allowed CPU count from `CPU_COUNT(sched_getaffinity())`. Use the allowed count as the default thread count. The two values may differ under `cpuset`.

6.2 Required Experiments

Keep the engine running past the final scheduled change. Increase `-r` if necessary; report the value used.

```
sudo ./cpumgr --bg 100 --weight 100 \  
  --at 10:weight=20 --at 20:weight=400 \  
  -- ./engine -n N -t 2 -m block -b 256 -r 50 -P  
  
sudo ./cpumgr --max 50000 100000 \  
  --at 10:max=20000,100000 --at 20:max=max,100000 \  
  -- ./engine -n N -t 2 -m block -b 256 -r 50 -P  
  
sudo ./cpumgr --cpuset 0 \  
  -- ./engine -n N -m block -b 256 -r 20 -P  
sudo ./cpumgr --cpuset 0 \  
  -- ./engine -n N -t 4 -m block -b 256 -r 20 -P
```

Report interval measurements, not only whole-run totals. Include all three `cpu.stat` fields and a source-log path.

Configuration	Prediction	Blocks/s	cpu.stat	run_delay	Interpretation
weight 100 -> 20 -> 400; bg 100					
quota 50% -> 20% -> unlimited					
cpuset 0; default threads					
cpuset 0; 4 threads					

6.3 Required Explanation

5. With background weight 100, expected CPU shares under contention are 1:1, 1:5, and 4:1 as the engine weight changes. Did interval CPU time and blocks/s follow this trend? Explain deviations, including idle time or hypervisor interference.
6. What happened to blocks/s and `nr_throttled` with `cpu.max=20000 100000`? Explain why the engine can be throttled even while CPUs are idle, how the 100ms period replenishes its budget, and why `run_delay` can differ from the weight experiment.
7. Under `cpuset 0`, what were the online and allowed CPU counts? Which should determine thread-pool size? How did `run_delay` and blocks/s change with four threads on one CPU?
8. Give two situations where changing CPU policy without changing application code is appropriate. Give one where changing block size or counter layout is more appropriate.

7. Level 3 - Quantum-Based User-Space Scheduler

7.1 Required Implementation

- Read jobs as arrival_ms base_blocks weight name. A base block is always 256KB, independent of -b; weight is used only by wrr. Enqueue jobs at arrival and process their records using work_block(), including a partial final block when needed.
- Workers select jobs according to the policy and run one block at a time. At each block boundary, measure CPU time used in the current turn with CLOCK_THREAD_CPUTIME_ID, not wall time.
- When CPU time reaches the quantum, save progress and requeue the unfinished job. Workers with no ready work must sleep on a condition variable. Busy waiting is prohibited.

Policy	Required Behavior
fifo	Run the selected job to completion without requeueing.
rr	Use quantum_ms, then move the unfinished job to the queue tail.
wrr	Use quantum_ms x weight / 100, then move the unfinished job to the queue tail.

- For each job, record arrival, first start, completion, response time (completion - arrival), CPU time, and interruption count. Separately accumulate CPU time for queue/bookkeeping and work_block(). Output a summary table and CSV.

7.2 Required Job Workload

- Two long jobs arrive at t=0 with weights 100 and 300. They must remain unfinished until the last short job arrives. Start with 10000 base blocks (256KB each) and adjust the long-job lengths for your machine.
- Ten short jobs arrive from t=1 second at 500ms intervals. Each uses 4 base blocks (1MB). Keep job-file lengths unchanged when comparing block sizes; only -b changes. Process a partial final block when necessary.

7.3 Required Experiments

```
for p in fifo rr wrr; do ./usched -p $p -q 20 -b 256 -w 1 -j jobs.txt -o p-$p.txt; done
for q in 2 5 20 100; do ./usched -p rr -q $q -b 256 -w 1 -j jobs.txt -o q-$q.txt; done
for b in 16 256 4096; do ./usched -p rr -q 20 -b $b -w 1 -j jobs.txt -o b-$b.txt; done
./usched -p wrr -q 20 -b 256 -w 2 -j jobs.txt -o wrr-w2.txt
```

Report overhead %, p50/p99 definitions, and source logs. Turn traces are saved as <output>.turns.csv.

Configuration	Prediction	Short-Job p50/p99	Long-Job CPU Share	Interruptions / Overhead %
fifo / rr / wrr; q=20ms				
rr; q=2/5/20/100ms				
rr; blocks=16KB/256KB/4MB				
wrr; 2 workers				

7.4 Required Analysis

9. Compare short-job p99 response time under fifo and rr. Why can a short job wait for a long job under FIFO? Which property does this share with the kernel FIFO scheduling class?
10. How did short-job response time and overhead % change from 2ms to 100ms quantum? Where did overhead stop falling substantially? Relate this to time slices and context-switch cost.
11. Block size sets the minimum safe preemption unit. Could long jobs using 4MB blocks meet a 20ms quantum? Report measured overshoot and explain why block boundaries are safe preemption points. Short jobs process only 1MB.
12. While both long jobs were runnable, did their CPU allocation follow the 1:3 weight ratio? Use turn traces over a common time interval, not final whole-job CPU totals. What changes with two workers, and why might the ratio differ?
13. How is CPU-time accounting similar to Linux sum_exec_runtime? What would go wrong if time spent preempted by the kernel also consumed a job's quantum?

8. Evidence Required

Level	Implementation Evidence	Runtime / Measurement Evidence	Analysis Evidence
1	engine + work_block + counter variants	stage/block + block-size + counter comparisons; CPU time and switches	locality, coherence, false sharing
2	cpumgr + live changes + cleanup	interval blocks/s + cpu.stat + schedstat; online/allowed CPUs	weight, quota, throttling, cpuset
3	ready queue + FIFO/RR/WRR + block-boundary preemption	short-job p50/p99 + long-job CPU share + overhead	quantum, fairness, safe preemption, CPU time

9. Submission Package

- engine, cpumgr, and usched source code; Makefile; jobs.txt; and the original log directory.
- README with kernel version, vCPU count, cache sizes from lscpu, exact build/run commands, and highest completed level.
- Technical report (recommended 4-6 pages) organized as Level 1 / Level 2 / Level 3. Include predictions, controlled comparisons, measured results, and explanations.
- Level 1 stage/block, block-size, and counter-layout/update tables; Level 2 interval CPU-policy measurements; Level 3 response-time, CPU-share, and overhead tables.
- Selected runtime evidence and exact commands. Every measurement row must identify its raw log. Do not submit pages of unannotated raw output.
- A short AI-use statement if AI tools were used, including at least one generated suggestion or code issue that you verified, changed, or rejected.

10. Evaluation

Category	Weight	What Is Evaluated
Implementation	25%	identical aggregate results; correct live cgroup changes and cleanup; block-boundary preemption without busy waiting
Experiment & Measurement	35%	predictions, fixed parameters, PID/kernel logs, repeatability, and aligned interval samples
Technical Explanation	30%	reasoning from measured values about cache/coherence, weight/quota/cpuset, and quantum/overhead
Code Quality & Report	10%	check-off modifications, reproducibility, and concise evidence

Performance is not graded by absolute numbers across different hardware. The quality of the controlled comparison, the correctness of the measurement method, and the explanation of observed behavior are graded.

A program that appears to work but is not experimentally validated or correctly explained cannot receive full credit.

11. Final Check-Off Questions

The oral check-off takes approximately five minutes. Predict each result before making the change and running it.

- Change only padded-slot alignment to 8 bytes; keep padding and allocation unchanged. Predict and verify the effect.
- Use --at 5:max=10000,100000. Predict blocks/s and nr_throttled per second before running the experiment.
- Replace CLOCK_THREAD_CPUTIME_ID with CLOCK_MONOTONIC in usched. Predict what changes under competing CPU load from cpumgr --bg 100, then explain the result.
- Give short jobs weight 400 under wrr. Predict their p99 response time before showing the measurement.

Build it. Measure what changes. Explain why.